

GitLab → GitHub

# 迁移 (Migration) 实战培训

术语对照 · 迁移 (Migration) 重点 · 最佳实践 · Hands-on Lab 全流程演练

讲师：高老师 (认证培训师) | 时长：3 小时 (含实操) | 版本 2.0

# 课程目标

帮助正在使用 GitLab 的团队 (Team)，平滑、低风险地完成向 GitHub 的迁移 (Migration) 与协作 (Collaboration) 模式转换

1

## 建立术语 / 观念映射

对照 GitLab 与 GitHub 的核心概念，降低平台切换认知成本

2

## 掌握有差异部分的迁移 (Migration) 策略

仓库 (Repository)、Issue/MR、权限 (Permission) 架构、CI/CD 的可落地迁移 (Migration) 方案与命令

3

## 理解迁移 (Migration) 最佳实践

评估、策略、节奏、权限 (Permission) 安全 (Security)、验证 (Validation)、回滚，指导后续独立迁移 (Migration)

4

## 独立完成一次完整迁移 (Migration)

在实验环境 (Environment) 中走通 GitLab → GitHub 全流程

受众：GitHub Enterprise 管理员 · 开发 (Development) 团队 (Team) · DevOps 与资安团队 (Team) | 预备：具备 Git 基础、了解 GitLab 日常操作

# 课程安排（四模块）

对照 → 迁移（Migration） → 最佳实践 → 实操 为主线

| 模块                       | 核心内容                                                                                                            |
|--------------------------|-----------------------------------------------------------------------------------------------------------------|
| 第一部分：术语与观念对照             | 顶层组织（Organization）、仓库（Repository）、分支保护（Branch Protection）、MR/PR、CI/CD、项目（Project）管理、安全（Security）、权限（Permission） |
| 第二部分：有差异部分的迁移（Migration） | 数据映射、权限（Permission）架构、git mirror、gh gl2gh、Actions Importer、Webhook                                              |
| 第三部分：迁移（Migration）最佳实践   | 前置评估、策略选择、执行节奏、沟通变更、权限（Permission）安全（Security）、验证（Validation）、回滚                                                |
| 第四部分：Hands-on Lab        | 镜像（Mirror）迁移（Migration）、gh gl2gh 批量迁移（Migration）、权限（Permission）架构重构（独立完成一次迁移（Migration））                        |

# 目录

## 01 GitLab 与 GitHub 术语及观念对照

---

## 02 有差异部分的迁移

---

## 03 迁移 (Migration) 最佳实践

---

## 04 Hands-on Lab

---

## 05 附：GitLab 与 GitHub 产品对比

---

## PART 01

# GitLab 与 GitHub 术语及观念对照

---

目标：建立正确概念映射，降低平台切换的认知成本

# 平台定位哲学差异（先看全局）

『产品哲学』不同，决定了迁移（Migration）时要重建什么

| 维度                          | GitLab                                                                   | GitHub                                                                       |
|-----------------------------|--------------------------------------------------------------------------|------------------------------------------------------------------------------|
| 产品哲学                        | 一体化 DevSecOps 平台（单一产品覆盖 SCM+CI/CD+安全（Security）+合规（Compliance）+价值流）       | 以代码协作（Collaboration）为核心，能力模块化（Actions / Advanced Security / Copilot 多为独立付费项） |
| 安全（Security）能力归属            | Ultimate 层级内置（SAST/DAST/依赖（Dependency）/容器（Container）/密钥（Secret）扫描（Scan）） | Advanced Security 为独立附加产品（按活跃提交（Commit）者计费）                                  |
| AI 编码助手                     | GitLab Duo（Premium 起含额度）                                                 | GitHub Copilot（独立订阅）                                                         |
| 私有（Private）仓库（Repository）限制 | Free 私有（Private）组限 5 个授权（Authorization）用户                                | 所有层级均无限私有（Private）仓库（Repository）                                             |

⚠ 一句话记忆：GitLab=买 Ultimate 最划算；GitHub=核心免费、企业能力按需叠加

# 顶层组织 (Organization) 对照

组织 (Organization) 结构是第一层映射, 理解了它, 后续权限 (Permission) 与项目 (Project) 归属都顺了

## GitLab

- Group: 顶层组织 (Organization) (如 training)
- Subgroup: 子组 (Subgroup) (如 training/backend)
- Project: 具体仓库 (Repository) (含 Issues/MR/CI)
- 三级结构: Group / Subgroup / Project
- 成员管理: Group 层邀请, 向下继承
- 访问级别:  
Guest→Reporter→Developer→Maintainer→Owner

## GitHub

- Organization: 顶层组织 (Organization) (对应 Group)
- Team: 团队 (Team) (可映射 Subgroup, 支持嵌套)
- Repository: 具体仓库 (Repository) (对应 Project)
- 结构: Org / Team / Repository
- 成员管理: Org 邀请, Team 赋权
- 团队 (Team) 权限 (Permission): Read / Triage / Write / Maintain / Admin

△ Subgroup 无直接对应 → 用 Team 嵌套 + Org 权限 (Permission) 分层实现类似效果

# 仓库 (Repository) 与可见性 (Visibility) 对照

Project ↔ Repository, 但可见性 (Visibility) 与协作 (Collaboration) 周边概念有差异

## GitLab

- Project: 仓库 (Repository) + Issue/MR/Wiki/CI 一体
- Visibility: Private / Internal / Public
- Fork: 支持 (fork 独立 project)
- Star (收藏)、Watch (通知)
- Templates: 项目 (Project) / Issue/MR 模板
- Topics: 仓库 (Repository) 标签 (Label) (可搜索)

## GitHub

- Repository: 仓库 (Repository) 本体 (协作 (Collaboration) 在周边)
- Visibility: Private / Internal / Public
- Fork: 基于账号, PR 回源
- Star (收藏)、Watch (自定义事件)
- Templates: .github 下 Issue/PR 模板
- Topics: 仓库 (Repository) 标签 (Label) (增强发现)

⚠ Internal 可见性 (Visibility) : GitLab=同实例登录用户; GitHub=仅 Org/Enterprise 内部 (Internal) 可见, 需 Ent 层级

# 分支保护 (Branch Protection) 规则对照

保护主干的关键设置, 迁移 (Migration) 后须在 GitHub 重建

## GitLab Protected Branch

- 谁可 push / merge 分别控制
- Require approvals (最少批准 (Approve) 数)
- Allow force push (默认关)
- Pipeline must succeed (状态门)
- Code Owner approval (可选)
- Merge trains (队列合并 (Merge) , 避免冲突)

## GitHub Branch Protection

- Require a pull request before merge
- Required approvals (最少审查数)
- Require status checks to pass
- Require signed commits / linear history
- Require review from CODEOWNERS
- Dismiss stale approvals / restrict pushes

△ 迁移 (Migration) 映射: GitLab merge trains → GitHub 无原生等价; 用 CODEOWNERS+状态检查+线性历史近似

# 合并请求 (Merge Request) vs Pull Request

概念一致，但审查语义、设置位置不同

## GitLab Merge Request

- 源分支 (Branch) → 目标分支 (Branch) 的变更请求
- 模板: `.gitlab/merge_request_templates/`
- Approve / Request changes / Suggested
- 合并 (Merge) 方式: merge / squash / rebase (可配)
- Draft / WIP 标记 (进行中)
- 多审查者 + 批准 (Approve) 规则 (分支 (Branch) 级)

## GitHub Pull Request

- 源分支 (Branch) → 目标分支 (Branch) 的变更请求
- 模板: `.github/pull_request_template.md`
- Review: Approve / Request changes / Suggested
- 默认 squash merge; 可允许 merge/rebase
- Draft PR (进行中)
- Reviewers + CODEOWNERS 必审

⚠ 迁移 (Migration) 时 MR 会自动转 PR (含 reviewer/approver/状态), 但仅携带最新 diff, 线程讨论 (Discussion) 扁平化

# 代码审查（Code Review）机制深讲

审查是协作（Collaboration）质量的核心，两个平台都能做但要会配

| 机制             | GitLab                               | GitHub                                     |
|----------------|--------------------------------------|--------------------------------------------|
| 指定审查人          | Reviewer（可多人，分支（Branch）规则设默认）        | Reviewers / CODEOWNERS 自动指定                |
| 批准（Approve）    | 分支保护（Branch Protection）：最少 approvals | 分支保护（Branch Protection）：Required approvals |
| 建议改动           | Suggested changes（可一键应用）             | Suggested change（可一键提交（Commit））            |
| 必审人            | Code Owner approval（可选）              | Require review from CODEOWNERS             |
| 草稿（Draft）      | Draft / WIP 前缀                       | Draft PR 开关                                |
| 讨论（Discussion） | Resolve thread 后解除阻塞                 | Conversation resolution 状态检查               |

# CI/CD 对照（重点：语法完全不同）

GitLab CI 不会自动迁移（Migration），必须重写为 GitHub Actions

## GitLab CI

- .gitlab-ci.yml 定义 stages / jobs
- runner 标签 (Label) 指定执行机
- artifacts / cache / rules 原生支持
- 变量 (Variable) 在 CI/CD 设置中集中管理
- needs 定义 job 依赖 (Dependency) (DAG)
- 一体化: SCM + CI 同平台

## GitHub Actions

- .github/workflows/\*.yml 定义 workflow
- runs-on 指定 runner (hosted / self-hosted)
- 需 actions/cache、upload/download-artifact
- Secrets / Variables 集中管理
- needs / if 定义依赖 (Dependency) 与条件
- 用 actions-importer 辅助转换 (目标 ~80%)

△ 关键差异: YAML 语法体系不同; 可用 gh actions-importer 辅助, 但仍需人工复核变量 (Variable) / 密钥 (Secret) / 缓存/制品 (Artifact)

# CI/CD 概念映射详表

把 GitLab CI 的构造逐一找到 GitHub Actions 对应

| GitLab CI             | GitHub Actions                              |
|-----------------------|---------------------------------------------|
| stages                | jobs 的 needs / 默认并行（无全局 stages）             |
| job / script          | job / run（多行脚本）                             |
| image                 | container（job 级）或 runs-on 镜像（Mirror）        |
| variables             | env / secrets（密） / vars（组织（Organization）级）  |
| artifacts             | actions/upload-artifact + download-artifact |
| cache                 | actions/cache（key 命中）                       |
| rules / only / except | on: / if: 条件（分支（Branch）/事件）                 |
| retry / timeout       | retry / timeout-minutes                     |
| schedule              | on: schedule（cron）                          |

# 项目 (Project) 管理对照

Issue / Epic / Milestone 的映射

## GitLab

- Issue: 任务 (Task)、缺陷 (Bug)、讨论 (Discussion)
- Epic: 跨 Issue 的大计划 (多层)
- Milestone: 迭代 (Iteration) 里程碑 (Milestone)
- Label: 分类标签 (Label) (支持色)
- Weight: Issue 权重估算
- Time tracking: 耗时记录

## GitHub

- Issue: 任务 (Task)、缺陷 (Bug)、讨论 (Discussion)
- Milestone (跨仓库 (Repository)) / Project (看板 (Board))
- Milestone: 迭代 (Iteration) 里程碑 (Milestone)
- Label: 分类标签 (Label) (支持色)
- 无原生 weight (可用 Projects 字段)
- 无原生 time tracking

△ Epic 映射: GitHub 用 Milestone + Projects 跨仓库 (Repository) 聚合; 重量级组合用多 repo Project

# 看板 (Board) 对照

日常协作 (Collaboration) 的可视化面板

## GitLab Issue Board

- 按 Label 映射列 (可多块板)
- Scope: Group / Project / 指定
- 拖拽卡片改 Label / 指派
- 与 Issue / Milestone 联动
- Premium+: Swimlanes (按指派)

## GitHub Projects

- 跨仓库 (Repository) 聚合 (Project V2)
- 视图: Board / Table / Roadmap
- 自定义字段、自动化 (Workflow)
- 与 Issue / PR 双向联动
- 支持 Insights 图表

⚠ 迁移 (Migration) : Issue Board 不自动迁; 在 GitHub 建 Project, 用 Label 映射列, 重做自动化

# Wiki / Snippets / Releases / Packages 对照

迁移 (Migration) 时需找对替代方案, 避免功能『消失』

| 能力       | GitLab                                         | GitHub                                  |
|----------|------------------------------------------------|-----------------------------------------|
| Wiki     | ✅ 项目 (Project) 知识库 (Knowledge Base) (Markdown) | ✅ 需 Org+Enterprise 才能私有 (Private) Pages |
| 代码片段     | Snippets (项目 (Project) / 个人)                   | Gist (公开 (Public) 或私密片段)                |
| Releases | ✅ 含二进制资产                                       | ✅ 含二进制资产                                |
| Packages | ✅ Container/NPM/Maven 等                        | ✅ GitHub Packages                       |
| 设计稿      | Design Management                              | 无直接对应 → 外部工具+链接                         |
| 时间追踪     | ✅ 内置                                           | 需第三方 App / 手动                           |

# 安全 (Security) 能力对照

这是两平台最大的『归属』差异

## GitLab (内置)

- SAST / DAST (代码/运行时)
- 依赖 (Dependency) 扫描 (Scan) / 容器 (Container) 扫描 (Scan)
- 密钥 (Secret) 检测 (Secret Detection)
- 模糊测试 (Test) (Fuzz)
- 全部在 Ultimate 层级内置
- 安全 (Security) 面板统一展示漏洞 (Vulnerability)

## GitHub (附加产品)

- CodeQL (SAST, Dependabot 依赖 (Dependency) )
- Dependabot alerts / security updates
- Secret scanning (推送 (Push) 保护)
- Code scanning / supply-chain
- Advanced Security 独立附加 (按提交 (Commit) 者计费)
- Security tab 聚合展示

⚠ 迁移 (Migration) 提醒: 安全 (Security) 策略不会自动迁; 在 GitHub 启用 Advanced Security + 重写扫描 (Scan) 工作流 (Workflow)

# 权限（Permission）模型深讲

把 GitLab 角色（Role）准确落到 GitHub 的 Org / Team / Repo

| GitLab 角色（Role） | GitHub 团队（Team）权限（Permission） | 典型用途                                        |
|-----------------|-------------------------------|---------------------------------------------|
| Owner           | Org Owner / Admin             | 组织（Organization）与计费、策略全控                    |
| Maintainer      | Team Maintain / Admin         | 管理仓库（Repository）设置、分支保护（Branch Protection）  |
| Developer       | Team Write / Maintain         | 日常推送（Push）、开 PR                             |
| Reporter        | Team Triage / Read            | 查看、分类、评论（Comment） Issue                     |
| Guest           | Team Read                     | 仅只读浏览私有（Private）/内部（Internal）仓库（Repository） |

⚠ Member 迁移（Migration）：gh gl2gh 以 mannequin 占位身份（Identity）迁入，需 reclaim 认领后赋真实权限（Permission）

# 第一部分小结：观念差异

从『一体化平台』到『模块化生态』的思维转换

- ◆ GitLab 单一平台覆盖 SCM+CI/CD+安全 (Security) + 合规 (Compliance) ; GitHub 以代码协作 (Collaboration) 为核心, 安全 (Security) /AI 多为独立产品
- ◆ 权限 (Permission) 模型: GitLab 用 Access Level 五级; GitHub 用 Team 权限 (Permission) 五级 + Org 分层
- ◆ Subgroup 无直接对应, 用 Team 嵌套 + Org 权限 (Permission) 实现
- ◆ 迁移 (Migration) 不是『复制』, 而是『映射 + 重建』—— 搞清楚哪些能自动迁、哪些必须手写

## PART 02

# 有差异部分的迁移

讲清『不能直接搬』的部分，给出可落地的迁移（Migration）策略与架构映射

# 迁移 (Migration) 全景图 (端到端流程)

一条主线：评估 → 迁代码 → 迁元数据 → 迁 CI → 重构权限 (Permission) → 切换

① 前置评估  
仓库 (Repository)  
盘点/依赖 (Dependency)  
) 梳理

② 迁代码  
mirror 完整历史

③ 迁元数据  
gh gl2gh

④ 迁 CI  
Actions  
Importer

⑤ 重构权限 (Permission)  
Org/Teams

⑥ 切换  
归档+重定向

△ 代码与元数据可并行迁移 (Migration) ; CI 与权限 (Permission) 重构可在验证 (Validation) 后再切换, 降低风险

# 迁移 (Migration) 范围与数据映射

两条路径：镜像 (Mirror) 搬代码、gh gl2gh 搬元数据

| 内容                                                   | 迁移 (Migration) 方式                     | 是否自动    |
|------------------------------------------------------|---------------------------------------|---------|
| commit / branch / tag 历史                             | git clone --mirror + push --mirror    | ✓ 完整保留  |
| Issue / MR / Label / Wiki / Milestone                | gh gl2gh (GitHub Enterprise Importer) | ✓ MR→PR |
| Releases / 评论 (Comment) / 表情回应                       | gh gl2gh                              | ✓       |
| CI/CD 流水线 (Pipeline) / 变量 (Variable) / 制品 (Artifact) | 需重写为 GitHub Actions                   | ✗ 手动    |
| Webhooks / Boards / Snippets                         | 需重建 (Projects / Gist)                 | ✗ 手动    |

# 保真度矩阵：能自动迁 vs 必须重建

迁移 (Migration) 前就讲清预期，避免学员『以为丢了』

## ✅ 自动迁移 (Migration) (高保真)

- Git 提交 (Commit)、分支 (Branch)、标签 (Label) (完整哈希)
- Issue (含评论 (Comment)、标签 (Label)、指派)
- MR → PR (reviewer/approver/状态)
- Wiki / Milestone / Release 资产
- 成员以 mannequin 占位身份 (Identity)

## ❌ 必须重建 (手动)

- CI/CD 流水线 (Pipeline) (语法不同)
- Webhooks、集成 (Integration)、Packages
- Issue Boards / Snippets / 时间追踪
- Group 设置 / 成员真实映射
- merge trains / required approvals 策略

⚠ mannequin 是占位账号，迁移 (Migration) 后由真实用户 reclaim 认领，再按映射赋权

# 权限 (Permission) 架构映射 (重点)

把 GitLab 的角色 (Role) 体系准确落到 GitHub 的 Org / Team / Repo

## GitLab 角色 (Role)

- Owner → Org Owner (组织 (Organization) 最高权限 (Permission) )
- Maintainer → Team Maintain / Admin
- Developer → Team Write / Maintain
- Reporter → Team Triage / Read
- Guest → Team Read (私有 (Private) /内部 (Internal) 仓库 (Repository) )

## GitHub 实现

- Org Owner: 组织 (Organization) 与计费管理
- Team Maintain: 管理仓库 (Repository) 设置、分支保护 (Branch Protection)
- Team Write: 日常推送 (Push) 、开 PR
- Team Triage: 查看、分类、评论 (Comment) Issue
- Read: 仅只读浏览

△ Subgroup 无直接对应 → Team 嵌套 + Org 权限 (Permission) 分层; 成员以 mannequin 占位身份 (Identity) 迁移 (Migration) , 需 reclaim 认领

# 仓库 (Repository) 迁移 (Migration) 策略:

最稳妥的代码迁移 (Migration) 方式, 保留完整历史

| 步骤                      | 命令                                                 | 说明                                 |
|-------------------------|----------------------------------------------------|------------------------------------|
| 1 拉取裸镜像 (Mirror)        | <code>git clone --mirror &lt;gitlab-url&gt;</code> | 含所有分支 (Branch) / 标签 (Label) / refs |
| 2 建空目标仓                 | 在 GitHub 创建空 repo (不初始化)                           | 避免冲突的历史                            |
| 3 推送 (Push) 镜像 (Mirror) | <code>git push --mirror &lt;github-url&gt;</code>  | 全量推送 (Push) refs                   |
| 4 校验                    | 对比 branch/tag/提交 (Commit) 数                        | 确认完整一致                             |

⚠ 仅搬代码与历史; LFS 需额外 `git lfs fetch --all + push`; Issue/MR/Wiki 不在此步

# gh gl2gh 命令速览

GitLab → GitHub 元数据迁移 (Migration) (2026 年 GA)

# 安装扩展 (2026 GA)

```
gh extension install gh-migration-tools/gh-gl2gh
```

# 设置凭证 (经典 PAT)

```
export GITLAB_PAT=<api,read_repository>    export GH_PAT=<repo,workflow,read:org>
```

# 单仓库迁移

```
gh gl2gh migrate-repo \  
  --gitlab-server-url https://gitlab.com \  
  --gitlab-group training/backend --gitlab-project demo-service \  
  --github-org DEST --github-repo demo-service --use-github-storage
```

⚠ GitLab PAT 需 api+read\_repository; GitHub 须 classic PAT (repo,workflow,read:org) ; 目标仅支持 Enterprise Cloud

# gh gl2gh 实战参数详解

批量迁移 (Migration) 与脚本生成, 适合企业级迁移 (Migration)

| 参数 / 命令                                                                                | 作用                                           |
|----------------------------------------------------------------------------------------|----------------------------------------------|
| --target-instance-type github-enterprise-cloud                                         | 指定目标类型 (必填)                                  |
| --use-github-storage                                                                   | 用 GitHub 存储 (Storage) 中转大仓库 (Repository) /附件 |
| generate-script                                                                        | 生成批量迁移 (Migration) 脚本 (多仓库 (Repository) 循环)  |
| --gitlab-group / --github-org                                                          | 源组 → 目标 Org 映射                               |
| gh gl2gh migrate-user --users                                                          | 批量迁移 (Migration) 用户为 mannequin               |
| ⚠ 企业批量: 先用 generate-script 产出脚本, dry-run 验证 (Validation) 后再正式跑; 目标仅支持 Enterprise Cloud |                                              |

# CI/CD 重建: GitHub Actions Importer

把 .gitlab-ci.yml 辅助转为 workflow, 再人工补全

## 流程

- gh extension install github/gh-actions-importer
- gh actions-importer configure (填凭证)
- gh actions-importer audit gitlab (盘点)
- gh actions-importer dry-run gitlab (生成)
- gh actions-importer migrate gitlab (开 PR)

## 人工复核清单

- 变量 (Variable) / 密钥 (Secret) → Secrets / Variables
- Runner 标签 (Label) → hosted / self-hosted
- 跨 job 缓存 → actions/cache
- 制品 (Artifact) → upload/download-artifact
- rules/only → needs / if / on

⚠ 目标转换率约 80%, 复杂流水线 (Pipeline) 需补写; GitLab 特定构造 (masked 变量 (Variable)、制品 (Artifact) 报告) 须手动迁移 (Migration)

# CI 概念映射表 (.gitlab-ci.yml → workflow)

动手前的对照速查

| GitLab CI             | GitHub Actions                             |
|-----------------------|--------------------------------------------|
| stages: [build, test] | 默认 job 并行; needs: 定义顺序                     |
| job: build / script:  | jobs: { build: { runs-on, steps: [run] } } |
| image: node:20        | runs-on: ubuntu-latest 或 container:        |
| artifacts: paths      | actions/upload-artifact@v4                 |
| cache: key            | actions/cache@v4 key:                      |
| rules: - if: \$CI_... | on: push / if: \${{ ... }}                 |
| schedule: cron        | on: schedule: - cron: '0 0 * * *'          |

# Webhooks / Secrets / LFS 重建

容易被遗漏但生产（Production）必需的收尾项

## 需在 GitHub 重建

- Webhooks: 重新配置 URL + 事件
- Secrets / Variables: 集中重设
- LFS: `git lfs fetch --all + push --all`
- Packages: 重新发布 (Release) 到 GitHub Packages
- 外部集成 (Integration) : Jira / Slack / 监控

## 注意事项

- Webhook Secret 重新生成, 避免泄漏
- 密文用 Vault / GitHub Secrets, 禁明文
- LFS 大文件超配额需评估存储 (Storage) 成本
- Packages 迁移 (Migration) 注意命名空间变化
- 通知类集成 (Integration) 需重新订阅目标 repo

⚠ 这些项 `gh gl2gh` 不搬运; 遗漏会导致『迁移 (Migration) 后 CI 跑不起来 / 通知断链』

# 切换与重定向策略

降低切换阵痛，保证历史链接可用

| 动作                                    | 做法                                 |
|---------------------------------------|------------------------------------|
| 归档 GitLab                             | 设为只读 Archive，保留历史访问                |
| 重定向                                   | 反向代理 / DNS 指向 GitHub               |
| 提交（Commit）说明改写                        | sed 批量替换旧 URL 引用                   |
| 文档更新                                  | README / CI 脚本 / 内部（Internal） Wiki |
| 公告                                    | 提前沟通切换窗口与影响                        |
| ⚠ 建议保留 GitLab 只读 1-2 个月，便于回溯与排错，再彻底下线 |                                    |

# 迁移 (Migration) 风险与回滚

提前预案，迁移 (Migration) 才稳

| 风险                 | 应对 / 回滚                                   |
|--------------------|-------------------------------------------|
| CI 未跑绿             | 保留 GitLab CI 并行运行，验证 (Validation) 后再切     |
| 权限 (Permission) 错配 | 对照映射表预演，切换前 dry-run 校验                    |
| 历史不完整              | mirror 前后提交 (Commit) 数比对，差异即回退            |
| 密钥 (Secret) 泄漏     | 旧 PAT 立即吊销，新 Secrets 重建                   |
| 大仓超时               | 分片迁移 (Migration) / 用 --use-github-storage |

## PART 03

# 迁移 (Migration) 最佳实践

把一次性迁移 (Migration) 变成可复制的方法论

# 最佳实践 1：迁移（Migration）前置评估

先摸清家底，再定方案

| 评估项                           | 要点                                                                          |
|-------------------------------|-----------------------------------------------------------------------------|
| 仓库（Repository）盘点              | 数量、大小、活跃度、LFS 资产、子系统依赖（Dependency）                                          |
| 工具链梳理                         | CI 变量（Variable）/密钥（Secret）、Webhooks、外部集成（Integration）、包仓库（Package Registry） |
| 权限（Permission）与合规（Compliance） | 谁有哪些权限（Permission）、敏感分支（Branch）、审计（Audit）要求                                 |
| 团队（Team）就绪度                   | Git 熟练度、培训需求、停机容忍度                                                          |

△ 产出物：迁移（Migration）清单（仓库（Repository）×内容×负责人）+ 风险登记册

## 最佳实践 2：迁移 (Migration) 策略选择

大爆炸 vs 渐进式，按风险选

### 大爆炸 (Big Bang)

- 一次性切换全部仓库 (Repository)
- 周期短、沟通简单
- 适合：小团队 (Team) / 低耦合
- 风险：回滚成本高
- 停机窗口：需要

### 渐进式 (Phased / Pilot)

- 先试点团队 (Team)，再分批推广
- 风险可控、学习曲线平滑
- 适合：中大型 / 多团队 (Team)
- 风险：双平台并行成本
- 停机窗口：可零停机

△ 推荐：先 pilot 1-2 个仓库 (Repository) 跑通全流程，沉淀手册后再批量推广

# 最佳实践 3：执行节奏与里程碑（Milestone）

示例 4 周计划（可按规模伸缩）

| 周次 | 重点                                   | 里程碑（Milestone）              |
|----|--------------------------------------|-----------------------------|
| W1 | 前置评估 + 试点仓库（Repository）迁移（Migration） | 试点仓库（Repository）在 GitHub 跑通 |
| W2 | 批量迁移（Migration） + CI 重建              | Actions 流水线（Pipeline）绿灯     |
| W3 | 权限（Permission）重构 + 数据验证（Validation）  | 权限（Permission）校验通过          |
| W4 | 切换 + 重定向 + 收尾                        | 旧 GitLab 只读归档               |

## 最佳实践 4：沟通与变更管理

技术之外，人是成功关键

- 干系人地图：管理层（价值）、开发（Development）（流程变化）、资安（合规（Compliance））分别沟通
- 培训计划：迁移（Migration）前做平台对照 workshop（即本课程内容）
- 停机窗口沟通：提前公告切换时间与影响范围
- 内部（Internal）FAQ + 迁移（Migration）手册下发，降低支持工单
- 设立迁移（Migration）支持群 / 值班，迁移（Migration）周实时答疑

△ 变更阻力常来自『不熟悉』——对照培训 + 试点体验是最有效的消解手段

# 最佳实践 5：权限 (Permission) 与安全 (Security)

最小权限 (Permission) + 可审计 (Audit)

- 最小权限 (Permission) 原则：用 Teams 授权 (Authorization)，避免个人直接赋权
- 分支保护 (Branch Protection) 规则 + CODEOWNERS 指定必审人，防止误合主干
- Secrets 移到 GitHub Secrets / Vault，禁用明文与日志打印
- 启用 2FA、IP 允许列表、审计 (Audit) 日志流 (Enterprise)
- 私有 (Private) 依赖 (Dependency) 用 Dependabot / CodeQL 做安全 (Security) 左移

△ 映射落地：Developer→Write/Maintain；Reporter→Triage/Read；Owner→Admin

## 最佳实践 6: CI/CD 重建

先盘点、再转换、后替换

- 先用 actions-importer audit 盘点, dry-run 预览生成结果
- 敏感变量 (Variable) → encrypted secrets; 环境 (Environment) 差异用 Environments
- 自托管 (Self-hosted) runner 安全 (Security) 基线: 无特权容器 (Container)、网络 (Network) 隔离、定期轮换
- 保留 GitLab CI 并行运行, 直到 Actions 跑绿再切换
- 复用 Marketplace 官方 Action, 减少自维护脚本

⚠ 目标转换率约 80%; 复杂 job 需人工补全条件/缓存/制品 (Artifact) 映射

# 最佳实践 7：数据完整性与验证 (Validation)

用脚本批量校验，不靠肉眼

| 检查项                                             | 确认方式                                                             |
|-------------------------------------------------|------------------------------------------------------------------|
| 代码与历史完整                                         | 分支 (Branch) / 标签 (Label) / 提交 (Commit) 数对比一致 (git rev-list 计数)   |
| Issue / PR / Label / Wiki / Milestone / Release | gh issue list / gh pr list 核对数量                                  |
| CI 运行通过                                         | GitHub Actions 跑绿                                                |
| 权限 (Permission) 正确                              | 多角色 (Role) 账号验证 (Validation)<br>Read/Triage/Write/Maintain/Admin |
| Webhook / Secrets / LFS                         | 触发测试 (Test) / clone 后文件完整                                        |

# 最佳实践 8：迁移（Migration）后优化

不止『搬过来』，更要『用起来』

- 利用生态：Copilot 辅助编码、Marketplace Actions 丰富流水线（Pipeline）
- Projects 看板（Board）跨仓库（Repository）聚合，统一管理多团队（Team）迭代（Iteration）
- Codespaces 标准化开发（Development）环境（Environment），降低『在我机器能跑』
- Discussions 替代零散沟通，知识沉淀可检索
- Dependabot / CodeQL 安全（Security）左移，融入 PR 审查

△ 迁移（Migration）是起点：GitHub 的模块化生态在稳定后可逐步释放效率红利

# 常见问题 FAQ

学员最易卡住的几个点

| 问题                    | 回答                                   |
|-----------------------|--------------------------------------|
| CI 为什么不能自动迁?          | YAML 语法体系不同, 需重写 Actions             |
| Subgroup 怎么办?         | 用 Team 嵌套 + Org 权限 (Permission) 分层实现 |
| gh gl2gh 支持 Server 吗? | 仅支持 Enterprise Cloud, 不支持 Server     |
| 历史会丢吗?                | mirror 完整保留提交 (Commit) 哈希            |
| 迁移 (Migration) 要停机吗?  | 可零停机 (GitLab 只读切换)                   |

## PART 04

# Hands-on Lab: 完成一次完整迁移

实验环境 (Environment) : GitHub Codespaces + gh gl2gh + GitHub Enterprise Cloud | 每位学员独立完成一次完整迁移 (Migration)

# Lab 总览：实验环境 (Environment) 与四个实验

配套《LAB-GitLab-to-GitHub-Migration.md》逐阶段演练，每个阶段都有 PASS 检查

## GitHub Codespaces

Linux Bash 终端，所有命令在终端执行，不要求本地 PowerShell

## gh gl2gh

GitHub Enterprise Importer 的 GitLab→GitHub 扩展 (2026 GA)

## Enterprise Cloud

目标组织 (Organization) 须为 GitHub Enterprise Cloud

## Lab 1

gh gl2gh 迁移 (Migration) 仓库 (Repository) 与元数据

## Lab 2

GitLab CI → GitHub Actions

## Lab 3

权限 (Permission) / CODEOWNERS / Ruleset

## Lab 4

Projects / Dependabot / 安全 (Security)

△ 准备阶段前置：Codespace / 凭据 (Credential) / 参数 / GitLab 项目 (Project) / 盘点冻结；每个阶段都有 PASS 检查，未通过必须停在该阶段

## PART 05

# 附：GitLab 与 GitHub 产品对比

帮助学员理解『网页版（Web / SaaS） / 企业版（Enterprise）』差异（2026 资料）

# 产品定位哲学对比

先看全局，再看清层级

| 维度                          | GitLab                                                                   | GitHub                                                                       |
|-----------------------------|--------------------------------------------------------------------------|------------------------------------------------------------------------------|
| 产品哲学                        | 一体化 DevSecOps 平台（单一产品覆盖 SCM+CI/CD+安全（Security）+合规（Compliance）+价值流）       | 以代码协作（Collaboration）为核心，能力模块化（Actions / Advanced Security / Copilot 多为独立付费项） |
| 安全（Security）能力归属            | Ultimate 层级内置（SAST/DAST/依赖（Dependency）/容器（Container）/密钥（Secret）扫描（Scan）） | Advanced Security 为独立附加产品（按活跃提交（Commit）者计费）                                  |
| AI 编码助手                     | GitLab Duo（Premium 起含额度）                                                 | GitHub Copilot（独立订阅）                                                         |
| 私有（Private）仓库（Repository）限制 | Free 私有（Private）组限 5 个授权（Authorization）用户                                | 所有层级均无限私有（Private）仓库（Repository）                                             |

# GitLab 产品层级 (2026)

三种部署 (Deploy) × 三个付费层级

| 层级       | 价格 (约)         | 关键能力                                                                                                           | 适合谁                                 |
|----------|----------------|----------------------------------------------------------------------------------------------------------------|-------------------------------------|
| Free     | \$0            | SCM+CI/CD、基础安全 (Security) (SAST)、5 私有 (Private) 授权 (Authorization) 用户/组、400 分钟、10GiB 存储 (Storage)              | 个人、开源 (Open Source)、试用              |
| Premium  | \$29/用户/月      | 无限用户、10,000 分钟、\$12 Duo 额度、高级 CI/CD、SLA、优先支持                                                                   | 成长型多团队 (Team) 组织 (Organization)     |
| Ultimate | 定制(估 \$99–149) | Premium 全部 + 应用安全 (Security) 测试 (Test)、供应链安全 (Security)、漏洞 (Vulnerability) 管理、合规 (Compliance) 治理、VSM、50,000 分钟 | 需安全 (Security) 与合规 (Compliance) 的企业 |

△ 自托管 (Self-hosted) 与 SaaS 同层级功能基本一致；少量差异 (如自托管 (Self-hosted) 含 LDAP Group Sync)

# GitHub 产品层级 (2026)

账户层 / 组织 (Organization) 层 / Enterprise 部署 (Deploy)

| 层级                         | 价格 (约)                 | 关键能力                                                                        | 适合谁                                 |
|----------------------------|------------------------|-----------------------------------------------------------------------------|-------------------------------------|
| Free (个人)                  | \$0                    | 无限私有 (Private) 仓库 (Repository) (受限)、2000 分钟、500MB 包、Dependabot 告警           | 个人/开源 (Open Source) 学习              |
| Pro (个人)                   | \$4/用户/月               | Free 全部 + 必需审查者、保护分支 (Branch) 、Code Owners、Wiki 、Pages                      | 独立开发 (Development) 者                |
| Team (组织 ( Organization) ) | \$4/用户/月               | Free 组织 (Organization) 全部 + 团队 (Team) 审查者、安全 ( Security) 概览、可选 Adv Security | 协作 (Collaboration) 型团队 (Team) (5+)  |
| Enterprise Cloud           | 从 \$21/用户/月            | Team 全部 + SAML/SCIM、EMU、审计 (Audit) 流、IP 允许、50,000 分钟、99.9% SLA              | 需身份 (Identity) /合规 (Compliance) 的企业 |
| Enterprise Server          | 自托管 (Self-hosted) (定制) | 与 Cloud 同企业能力，运行在你自己的硬件/云，自行运维                                              | 须数据完全自托管 (Self-hosted)              |

# 部署（Deploy）形态细化对比

Web 版 / 自托管（Self-hosted） / 企业云的异同

| 对比项                         | GitLab.com | GitLab SM/Dedicated         | GitHub.com | GitHub Ent Cloud/Server |
|-----------------------------|------------|-----------------------------|------------|-------------------------|
| 谁运维                         | GitLab     | 你/Dedicated                 | GitHub     | GitHub(Cloud)/你(Server) |
| 数据驻留                        | GitLab 云   | 可选区域(Dedicated)             | GitHub 云   | 可选区域(Cloud)/自定(Server)  |
| 身份（Identity）集成（Integration） | 基础         | LDAP/AD(SM)、SAML(Dedicated) | 基础         | SAML/SCIM、EMU(Ent)      |
| 合规（Compliance）/审计（Audit）    | 有限         | 完整(审计（Audit）日志/SoD)         | 有限         | 完整(审计（Audit）流/IP 规则集)   |
| 高可用/容灾                      | GitLab 负责  | Geo(SM)/内置(Dedicated)       | GitHub 负责  | GitHub 负责(Cloud)        |
| 迁移（Migration）目标             | 可作源/目标     | 可作源/目标                      | 可作源/目标     | gl2gh 仅支持 Cloud         |

# 功能维度对比矩阵

两个平台核心能力逐项对照（Ultimate / Enterprise 代表）

| 功能维度                                 | GitLab（Ultimate 代表）                                  | GitHub（Enterprise 代表）                          |
|--------------------------------------|------------------------------------------------------|------------------------------------------------|
| 源代码管理（SCM / Source Code Management）  | Group/Subgroup/Project                               | Org/Team/Repository                            |
| 合并请求（Merge Request） / 审查             | MR（Approve / Request changes）                        | PR（Approve / Request changes / Suggested）      |
| CI/CD                                | GitLab CI（原生一体化）                                     | GitHub Actions（独立、生态丰富）                        |
| 安全扫描（Security Scan）                  | 内置 SAST/DAST/依赖（Dependency）/容器（Container）/密钥（Secret） | Advanced Security（独立附加）                        |
| 项目（Project）管理                        | Issue / Epic / Milestone / Board                     | Issues / Projects(跨仓库（Repository）) / Milestone |
| Wiki                                 | ✔                                                    | ✔（需 Org+Enterprise 才能私有（Private）Pages）         |
| Packages                             | ✔ Container/NPM 等                                    | ✔ GitHub Packages                              |
| Releases                             | ✔ 含资产                                                | ✔ 含资产                                          |
| Snippets                             | ✔                                                    | Gist（替代）                                       |
| 价值流分析（VSM / Value Stream Management） | ✔ VSM（Ultimate）                                      | Insights（有限）                                   |
| 企业身份（Enterprise Identity）            | LDAP/SAML(SM/Dedicated)                              | SAML/SCIM/EMU(Enterprise)                      |
| SLA                                  | Premium+（优先级支持）                                      | Enterprise Cloud 99.9%                         |
| AI 助手                                | GitLab Duo                                           | Copilot（独立）                                    |

# 典型选型建议

按团队 (Team) 规模与合规 (Compliance) 需求选择

## 个人 / 开源 (Open Source) / 学习

GitHub Free 或 GitLab Free; GitHub 私有 (Private) 仓库 (Repository) 无限更友好

## 中小团队 (Team) 要少工具

GitLab Premium 常比 GitHub Ent+Copilot 更省 (已含 Duo)

## 大型企业要合规 (Compliance)

GitLab Ultimate 或 GitHub Ent Cloud + Advanced Security + Copilot

## 强数据驻留 / 自托管 (Self-hosted)

GitLab Dedicated 或 GitHub Enterprise Server

# 迁移 (Migration) 视角：哪些层级能力需重建

呼应课程——无论选哪一层级，这些都不会自动带走

- CI/CD 流水线 (Pipeline)、变量 (Variable)、制品 (Artifact)、调度 (用 Actions Importer 或手写重建)
- Webhooks、Issue Boards、Snippets、时间追踪、Design Management
- Group 设置与成员关系 (用 Org / Teams 重建)
- Repo 策略 (merge trains、required approvals、topics) → 用 Branch protection / CODEOWNERS

△ 教学提示：gh gl2gh 当前仅支持目标为 Enterprise Cloud；课程 Demo 请用 Enterprise Cloud 沙盒 Org

# 课程总结

三句话带走 + 一份方法论

- 1 对照先行：GitLab 与 GitHub 概念——映射，Subgroup 用 Team 嵌套实现
- 2 迁移（Migration）有边界：代码/历史/Issue/PR 能自动迁；CI/Webhook/Board/Snippets 必须重建
- 3 实操为王：镜像（Mirror） + gh gl2gh + Actions Importer + 权限（Permission）重构，按验证（Validation）清单收尾
- 4 方法论沉淀：评估→策略→节奏→验证（Validation）→回滚，让下一次迁移（Migration）可复制